

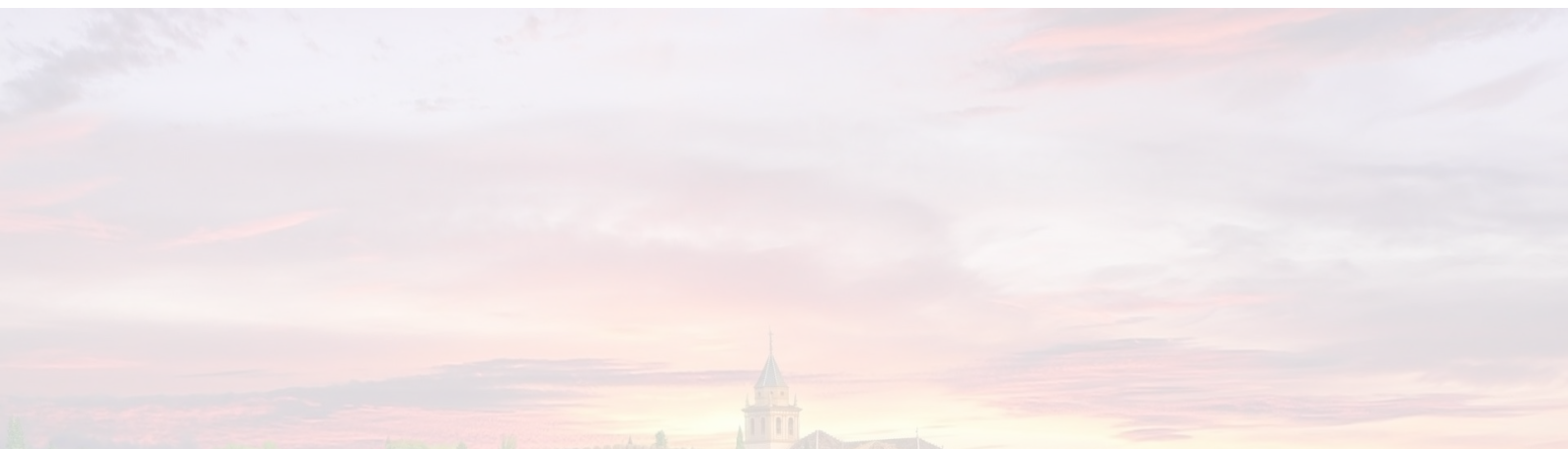


Ingeniería Informática + ADE

Universidad de Granada (UGR)

Autor: Ismael Sallami Moreno

Asignatura: 2º Parcial SCD



Índice

1. Examen Carlos Ureña 2014-2015	3
1.1. Enunciado	3
1.1.1. Ejercicio 1. Corrección de errores (6 puntos)	3
1.1.2. Ejercicio 2. Problema de múltiples Productores-Consumidores con Paso de Mensajes (4 puntos)	3
1.1.3. Plantilla	4
1.2. Solución Carlos Ureña 2014-2015	7
2. Examen de Mario Bros con Yoshi como NPC	16
2.1. Enunciado	16
2.2. Solución	17
3. Examen de Consumidores (valores pares e impares)	21
3.1. Enunciado	21
3.2. Solución	22
3.2.1. Partimos de la plantilla	22
3.2.2. Solución	26
4. Examen de modificar camarero y filósofos	31
4.1. Enunciado	31
4.2. Solución	32
4.2.1. Partimos de la plantilla	32
4.2.2. Solución	35
5. Examen Realizado del grupo A3 de este curso	39
5.1. Enunciado	39
5.2. Solución	39
6. Materiales	42

1 Examen Carlos Ureña 2014-2015

1.1. Enunciado

Asignatura: Sistemas Concurrentes y Distribuidos.

Año Académico: 2014/2015.

Grado: Doble Grado en Ingeniería Informática y ADE + Matemáticas.

Descripción: Examen correspondiente a la práctica 3 de SCD.

Profesor: Carlos Ureña.

Fecha: 12 de enero de 2015.

1.1.1. Ejercicio 1. Corrección de errores (6 puntos)

Considerar el código del archivo disponible en la subsection de plantilla de este documento o aquí, que intenta plantear una solución al problema de la cena de los filósofos con camarero. Hay diversos errores en el código proporcionado que deberás solucionar:

1. Intenta compilar el código proporcionado. Comprueba que hay diversos errores básicos de sintaxis que impiden que el código genere un ejecutable. Encuentra dichos errores y arrégloslos hasta que el código compile correctamente. Indica en el folio del examen el número de línea y cómo debería quedar ésta una vez solventado el problema.
2. Ejecuta el programa y observa la salida que genera. El algoritmo presenta un error en su diseño. Explica brevemente en el folio del examen por qué la salida proporcionada no es correcta.
3. Observa el código del programa ejecutado. Encuentra y corrige el error en el diseño del algoritmo que impide que el programa muestre una salida coherente. Señala en el folio del examen en qué línea está el problema y cómo debería quedar ésta para solucionar el error.
4. Se desea modificar el algoritmo dado del problema de los filósofos con camarero para dar servicio a 7 filósofos y 7 tenedores en lugar de a 5. También se desea cambiar el identificador de los procesos para que los filósofos sean los procesos con rank 0, 1, 2, ..., 6 y los tenedores sean los procesos 7, 8, ..., 13. Describe brevemente qué cambios debes realizar en el código proporcionado y en qué líneas.

1.1.2. Ejercicio 2. Problema de múltiples Productores-Consumidores con Paso de Mensajes (4 puntos)

Partiendo del algoritmo desarrollado durante la Práctica 3 para resolver el problema de múltiples Productores-Consumidores usando paso de mensajes MPI, se pide realizar los siguientes cambios:

- El valor que los procesos productores que envían al buffer debe ser un valor aleatorio entre 0 y 9 en lugar de uno secuencial como hasta ahora.

- Así mismo, el productor deberá enviar un mensaje al buffer pidiendo permiso para enviar el valor, y el buffer deberá responder al productor autorizando a enviar el mensaje antes de hacerlo (al igual que ya hace el consumidor).

Realiza los cambios necesarios para que, en el caso de que en el buffer se almacenen dos valores iguales consecutivos (independientemente del productor que los envíe), el buffer finalice.

- Antes de finalizar deberá avisar a todos los productores y consumidores para que estos finalicen enviando un valor especial en los mensajes de autorización (por ejemplo, un valor negativo). Sólo cuando todos los procesos hayan finalizado el buffer terminará.
- Implementa esta condición de forma que cuando cualquier proceso vaya a finalizar, imprima un mensaje en pantalla informando de ello.

1.1.3. Plantilla

```

1 #include <iostream>
2 #include <time.h>
3 #include <unistd.h>
4 #include <stdlib.h>
5 #include <mpi.h>
6
7 #define NUM_FILOSOFOS 5
8 #define NUM_PROCESOS (2*(NUM_FILOSOFOS)+1)
9 #define RANK_CAMARERO ((NUM_PROCESOS)-1)
10
11 #define TAG_SENTARSE 0
12 #define TAG_LEVANTARSE 1
13
14 using namespace std;
15 void Filosofo(int id);
16 void Tenedor(int id);
17 void Camarero();
18
19 // -----
20
21 void retraso_aleatorio()
22 {
23     static bool primera = true;
24
25     // inicializar generador de números aleatorios (la primera vez)
26     if (primera)
27     {
28         srand(time(NULL));
29         primera = false;
30     }
31     // retraso aleatorio, de entre una y dos décimas de segundo
32     usleep(1000L * (100L + rand() % 100L));
33 }
34
35 // -----

```



```

36
37 int main(int argc, char **argv)
38 {
39     int rank, size;
40
41     MPI_Init(&argc, &argv);
42     MPI_Comm_rank(MPI_COMM_WORLD, &rank);
43     MPI_Comm_size(MPI_COMM_WORLD, &size);
44
45     if (size != NUM_PROCESOS)
46     {
47         if (rank == 0)
48             cout << "El numero de procesos debe ser " << NUM_PROCESOS <<
49                 endl << flush;
50         MPI_Finalize();
51         return 0;
52     }
53
54     if (rank == RANK_CAMARERO)
55         Camarero(); // el 10 es el camarero
56     else if ((rank % 2) == 0)
57         Filosofo(rank); // los pares (0,2,4,6,8) son Filósofos
58     else
59         Tenedor(rank); // los impares (1,3,5,7,9) son Tenedores
60
61     MPI_Finalize();
62     return 0;
63 }
64 // -----
65
66 void Filosofo(int id)
67 {
68     const int
69         primero = (id + 1) % (NUM_PROCESOS - 1),
70         segundo = (id + (NUM_PROCESOS - 2)) % (NUM_PROCESOS - 1);
71
72     while (true)
73     {
74         // solicita sentarse
75         cout << "Filosofo " << id << " solicita sentarse." << endl << flush
76             ;
77         MPI_Ssend(&id, 1, MPI_INT, RANK_CAMARERO, TAG_SENTARSE,
78             MPI_COMM_WORLD);
79
80         // solicita y coge primer tenedor
81         cout << "Filosofo " << id << " solicita primer tenedor: " <<
82             primero << endl << flush;
83         MPI_Ssend(&id, 1, MPI_INT, primero, primero, MPI_COMM_WORLD);
84         cout << "Filosofo " << id << " coge primer tenedor: " << primero <<
85             endl << flush;
86
87         // solicita coge segundo tenedor
88         cout << "Filosofo " << id << " coge segundo tenedor: " << segundo
89             << endl << flush;
90         MPI_Ssend(&id, 1, MPI_INT, segundo, segundo, MPI_COMM_WORLD);

```

```

86     cout << "Filosofo " << id << " coge segundo tenedor: " << segundo
      << endl << flush;
87
88     // come
89     cout << "Filosofo " << id << " COMIENDO" << endl << flush;
90     retraso_aleatorio();
91
92     // suelta el segundo tenedor
93     cout << "Filosofo " << id << " suelta tenedor: " << segundo << endl
      << flush;
94     MPI_Ssend(&id, 1, MPI_INT, primero, primero, MPI_COMM_WORLD);
95
96     // suelta el primer tenedor
97     cout << "Filosofo " << id << " suelta tenedor: " << primero << endl
      << flush;
98     MPI_Ssend(&id, 1, MPI_INT, segundo, segundo, MPI_COMM_WORLD);
99
100    // solicita levantarse
101    cout << "Filosofo " << id << " solicita levantarse." << endl <<
      flush;
102    MPI_Ssend(&id, 1, MPI_INT, RANK_CAMARERO, TAG_LEVANTARSE,
      MPI_COMM_WORLD);
103
104    // piensa
105    cout << "Filosofo " << id << " PENSANDO" << endl << flush;
106    retraso_aleatorio();
107 }
108 }
109
110 // -----
111
112 void Tenedor(int id)
113 {
114     int valor;
115     MPI_Status status;
116
117     while (true)
118     {
119         // espera un mensaje desde cualquier filosofo vecino
120         cout << "Tenedor " << id << " espera petición." << endl << flush;
121         MPI_Recv(&valor, 1, MPI_INT, MPI_ANY_SOURCE, id, MPI_COMM_WORLD, &
            status);
122         cout << "Tenedor " << id << " recibe petición de " << status.
            MPI_SOURCE << endl << flush;
123
124         // espera a que el filosofo suelte el tenedor
125         MPI_Recv(&valor, 1, MPI_INT, status.MPI_SOURCE, id, MPI_COMM_WORLD,
            &status);
126         cout << "Tenedor " << id << " recibe liberación de " << status.
            MPI_SOURCE << endl << flush;
127     }
128 }
129
130 // -----
131
132 void Camarero()

```

```

133 {
134     int valor, num_sentados = 0, tag_aceptada;
135     MPI_Status status;
136
137     while (true)
138     {
139         if (num_sentados < NUM_FILOSOFOS - 1)
140             tag_aceptada = MPI_ANY_TAG;
141         else
142             tag_aceptada = TAG_LEVANTARSE;
143
144         // espera un mensaje desde cualquier filosofo con etiqueta
145         // aceptable
146         cout << "Camarero espera petición de sentarse/levantarse (sentados
147             == " << num_sentados << ")" << endl << flush;
148
149         MPI_Recv(&valor, 1, MPI_INT, MPI_ANY_SOURCE, tag_aceptada,
150             MPI_COMM_WORLD, &status);
151         cout << "Camarero recibe petición de filosofo " << status.
152             MPI_SOURCE << endl << flush;
153
154         if (status.MPI_TAG == TAG_LEVANTARSE)
155             num_sentados--;
156         else
157             num_sentados++;
158     }
159 }

```

1.2. Solución Carlos Ureña 2014-2015

Ejercicio 1

- Errores:
 - En la línea 84 falta un argumento en la función MPI_Ssend. No está puesto el tag del mensaje (debería ser "segundo").
 - En la línea 124 que los dos primeros argumentos están al revés.
- Error en el diseño: podemos ver que tras la ejecución del programa, los filósofos no se levantan de la mesa. El error se encuentra en la línea 101, en el paso del mensaje al camarero para solicitar levantarse, el tag del mensaje es el de sentarse (TAG_SENTARSE), en lugar del de levantarse (TAG_LEVANTARSE).
- Último apartado:
 - para dar servicio a 7 filósofos y 7 tenedores, se deben cambiar las constantes NUM_FILOSOFOS en la línea 7.
 - para ello debemos de cambiar la línea 54 poniendo que el rank debe de ser menor al número de filósofos.
 - además, debemos de cambiar la asignación de los tenedores en la línea 68 y 69, debido a que la presente no es coherente ni consistente desembocando en un error en el que varios filósofos quieren coger el mismo tenedor.

```

1 #include <iostream>
2 #include <time.h>
3 #include <unistd.h>
4 #include <stdlib.h>
5 #include <mpi.h>
6
7 #define NUM_FILOSOFOS 7
8 #define NUM_PROCESOS (2*(NUM_FILOSOFOS)+1)
9 #define RANK_CAMARERO ((NUM_PROCESOS)-1)
10
11 #define TAG_SENTARSE 0
12 #define TAG_LEVANTARSE 1
13
14 using namespace std ;
15 void Filosofo( int id );
16 void Tenedor ( int id );
17 void Camarero() ;
18
19 // -----
20
21 void retraso_aleatorio()
22 {
23     static bool primera = true ;
24
25     // inicializar generador de numeros aleatorios (la primera vez)
26     if ( primera )
27     { srand(time(NULL)) ;
28       primera = false ;
29     }
30     // retraso aleatorio, de entre una y dos décimas de segundo
31     usleep( 1000L * (100L + rand() % 100L ) );
32 }
33
34 // -----
35
36 int main(int argc, char** argv )
37 {
38     int rank, size;
39
40     MPI_Init( &argc, &argv );
41     MPI_Comm_rank( MPI_COMM_WORLD, &rank );
42     MPI_Comm_size( MPI_COMM_WORLD, &size );
43
44     if ( size != NUM_PROCESOS )
45     {
46         if( rank == 0 )
47             cout << "El numero de procesos debe ser " << NUM_PROCESOS << endl
48                 << flush;
49         MPI_Finalize( );
50         return 0;
51     }
52
53     if ( rank == RANK_CAMARERO )
54         Camarero() ; // el 14 es el camarero
55     else if (rank < NUM_FILOSOFOS)

```



```

55     Filosofo( rank );
56     else
57         Tenedor( rank );
58
59     MPI_Finalize( );
60     return 0;
61 }
62
63 // -----
64
65 void Filosofo( int id )
66 {
67     const int
68         primero = NUM_FILOSOFOS + id ,
69         segundo = NUM_FILOSOFOS + ((id+1)%NUM_FILOSOFOS) ;
70
71     while ( true )
72     {
73         // solicita sentarse
74         cout << "Filosofo " << id << " solicita sentarse." << endl << flush
75             ;
76         MPI_Ssend( &id, 1, MPI_INT, RANK_CAMARERO, TAG_SENTARSE,
77             MPI_COMM_WORLD );
78
79         // solicita y coge primer tenedor
80         cout << "Filosofo " << id << " solicita primer tenedor: " <<
81             primero << endl << flush;
82         MPI_Ssend( &id, 1, MPI_INT, primero, primero, MPI_COMM_WORLD );
83         cout << "Filosofo " << id << " coge primer tenedor: " << primero <<
84             endl << flush;
85
86         // solicita coge segundo tenedor
87         cout << "Filosofo " << id << " coge segundo tenedor: " << segundo
88             << endl << flush;
89         MPI_Ssend( &id, 1, MPI_INT, segundo, segundo, MPI_COMM_WORLD );
90         cout << "Filosofo " << id << " coge segundo tenedor: " << segundo
91             << endl << flush;
92
93         // come
94         cout << "Filosofo " << id << " COMIENDO" << endl << flush;
95         retraso_aleatorio() ;
96
97         // suelta el segundo tenedor
98         cout << "Filosofo " << id << " suelta tenedor: " << segundo << endl
99             << flush;
100         MPI_Ssend( &id, 1, MPI_INT, primero, primero, MPI_COMM_WORLD );
101
102         // suelta el primer tenedor
103         cout << "Filosofo " << id << " suelta tenedor: " << primero << endl
104             << flush;
105         MPI_Ssend( &id, 1, MPI_INT, segundo, segundo, MPI_COMM_WORLD );
106
107         // solicita levantarse
108         cout << "Filosofo " << id << " solicita sentarse." << endl << flush
109             ;
110         MPI_Ssend ( &id, 1, MPI_INT, RANK_CAMARERO, TAG_LEVANTARSE,

```

```

MPI_COMM_WORLD );
102
103 // piensa
104 cout << "Filosofo " << id << " PENSANDO" << endl << flush ;
105 retraso_aleatorio() ;
106 }
107 }
108
109 // -----
110
111 void Tenedor( int id )
112 {
113     int valor ;
114     MPI_Status status;
115
116     while ( true )
117     {
118         // espera un mensaje desde cualquier filosofo vecino
119         cout <<"Tenedor " << id << " espera petición." << endl << flush;
120         MPI_Recv( &valor, 1, MPI_INT, MPI_ANY_SOURCE, id, MPI_COMM_WORLD, &
            status );
121         cout << "Tenedor " << id << " recibe petición de " << status.
            MPI_SOURCE << endl << flush ;
122
123         // espera a que el filosofo suelte el tenedor
124         MPI_Recv( &valor, 1, MPI_INT, status.MPI_SOURCE, id, MPI_COMM_WORLD
            , &status );
125         cout<<"Tenedor " << id << " recibe liberación de " << status.
            MPI_SOURCE << endl << flush;
126     }
127 }
128
129 // -----
130
131 void Camarero()
132 {
133     int valor, num_sentados = 0, tag_aceptada ;
134     MPI_Status status;
135
136     while ( true )
137     {
138         if ( num_sentados < NUM_FILOSOFOS-1 )
139             tag_aceptada = MPI_ANY_TAG ;
140         else
141             tag_aceptada = TAG_LEVANTARSE ;
142
143         // espera un mensaje desde cualquier filosofo con etiqueta
            aceptable
144         cout << "Camarero espera petición de sentarse/levantarse (sentados
            == " << num_sentados << ")" << endl << flush;
145
146         MPI_Recv( &valor, 1, MPI_INT, MPI_ANY_SOURCE, tag_aceptada,
            MPI_COMM_WORLD, &status);
147         cout << "Camarero recibe petición de filosofo " << status.
            MPI_SOURCE << endl << flush ;
148

```

```

149     if ( status.MPI_TAG == TAG_LEVANTARSE )
150         num_sentados -- ;
151     else
152         num_sentados ++ ;
153 }
154 }

```

La solución en código se encuentra aquí.

Ejercicio 2

En la resolución de este ejercicio usamos esta (plantilla)(pincha en plantilla para acceder a ella).

- Hemos añadido en la línea 58 en la función de producir valor esta línea **int valor = aleatorio<0, 9>()**; para que produzca el valor aleatorio.
- En la función del productor debemos añadir **MPI_Ssend(&peticion, 1, MPI_INT, n_p, etiq_productor, MPI_COMM_WORLD)**; para que el productor pida permiso para enviar el valor y añadimos en el **switch** que cuando sea el caso del productor reciba la petición de aviso: **MPI_Recv(&peticion_productor, 1, MPI_INT, MPI_ANY_SOURCE, etiq_productor, MPI_COMM_WORLD, &estado)**;

Para la solución del apartado 2.2 revisa este código donde los comentarios **apartado2.2** reflejan los cambios que se han realizado.

```

1  #include <chrono> // duraciones (duration), unidades de tiempo
2  #include <cstdlib>
3  #include <iostream>
4  #include <mpi.h>
5  #include <random> // dispositivos, generadores y distribuciones aleatorias
6  #include <thread> // this_thread::sleep_for
7
8  using namespace std;
9  using namespace std::this_thread;
10 using namespace std::chrono;
11
12 #define TERM -1 // Valor especial para finalizar
13
14 const int
15     etiq_prod = 1, // Etiqueta para mensajes de productores
16     etiq_cons = 2, // Etiqueta para mensajes de consumidores
17     num_prod = 4, // Número de productores
18     num_cons = 5, // Número de consumidores
19     id_ini_productores = 0, // ID inicial de los productores
20     id_ini_consumidores = num_prod + 1, // ID inicial de los consumidores
21     id_buffer = num_prod, // ID del buffer
22     num_procesos_esperado = num_prod + num_cons + 1, // Número total de
        procesos esperados
23     num_items = 20, // Número total de ítems a producir/consumir
24     tam_vector = 10; // Tamaño del buffer
25
26 /**
27  * @brief Función que produce un número aleatorio entre dos valores

```

```

28 *
29 * @tparam min Valor mínimo. Constante conocida en tiempo de compilación
30 * @tparam max Valor máximo. Constante conocida en tiempo de compilación
31 */
32 template <int min, int max> int aleatorio() {
33     static default_random_engine generador((random_device())());
34     static uniform_int_distribution<int> distribucion_uniforme(min, max);
35     return distribucion_uniforme(generador);
36 }
37 // -----
38 /**
39 * @brief Función que simula la producción de un valor entre 0 y 9
40 *
41 * @note Duerme un tiempo aleatorio entre 10 y 100 milisegundos
42 *
43 * @return int Valor producido
44 */
45 int producir() {
46     sleep_for(milliseconds(aleatorio<10, 100>())); // Simula tiempo de
47     // producción
48     return aleatorio<0, 9>(); // Devuelve un valor aleatorio entre 0 y 9
49 }
50 /**
51 * @brief Función que simula la producción de valores por un productor
52 *
53 * @param orden Orden del productor (global)
54 */
55 void funcion_productor(int orden) {
56     for (unsigned int i = 0; i < num_items / num_prod; i++) {
57         // producir valor
58         int valor_prod = producir();
59         int rec;
60         MPI_Status estado;
61         // enviar valor
62         cout << "Productor va a enviar valor " << valor_prod << endl <<
63             flush;
64         MPI_Ssend(&valor_prod, 1, MPI_INT, id_buffer, etiq_prod,
65             MPI_COMM_WORLD); // Enviar valor al buffer
66         MPI_Recv(&rec, 1, MPI_INT, id_buffer, 0, MPI_COMM_WORLD, &estado);
67         // Recibir confirmación del buffer
68         if (rec == TERM) { // Si recibe el valor especial, termina
69             break;
70         }
71         MPI_Ssend(&valor_prod, 1, MPI_INT, id_buffer, 0, MPI_COMM_WORLD);
72         // Enviar valor al buffer
73     }
74     cout << "Termina productor " << orden << endl; // Mensaje de finalizaci
75     // ón del productor
76 }
77 // -----
78 /**
79 * @brief Función que simula la consumición de un valor
80 *
81 * @param valor_cons Valor a consumir

```

```

78  */
79 void consumir(int valor_cons) {
80     // espera bloqueada
81     sleep_for(milliseconds(aleatorio<110, 200>())); // Simula tiempo de
        consumo
82     cout << "Consumidor ha consumido valor " << valor_cons << endl << flush
        ;
83 }
84 // -----
85
86 /**
87  * @brief Función que simula la consumición de valores por un consumidor
88  *
89  * @param orden Orden del consumidor (global)
90  */
91 void funcion_consumidor(int orden) {
92     int peticion, valor_rec = 1;
93     MPI_Status estado;
94
95     for (unsigned int i = 0; i < num_items / num_cons; i++) {
96         MPI_Ssend(&peticion, 1, MPI_INT, id_buffer, etiq_cons,
97             MPI_COMM_WORLD); // Enviar petición al buffer
98         MPI_Recv(&valor_rec, 1, MPI_INT, id_buffer, 0, MPI_COMM_WORLD, &
99             estado); // Recibir valor del buffer
100         if (valor_rec == TERM) { // Si recibe el valor especial, termina
101             break;
102         }
103         cout << "Consumidor ha recibido valor " << valor_rec << endl <<
104             flush;
105         consumir(valor_rec); // Consumir el valor recibido
106     }
107     cout << "Termina consumidor " << orden << endl; // Mensaje de
108         finalización del consumidor
109 }
110
111 /**
112  * @brief Función que simula un buffer de tamaño tam_vector
113  */
114 void funcion_buffer() {
115     int buffer[tam_vector], // buffer con celdas ocupadas y vacías
116         valor, // valor recibido o enviado
117         primera_libre = 0, // índice de primera celda libre
118         primera_ocupada = 0, // índice de primera celda ocupada
119         num_celdas_ocupadas = 0; // número de celdas ocupadas
120     MPI_Status estado; // metadatos del mensaje recibido
121     bool terminar = false; // bandera para finalizar
122
123     for (unsigned int i = 0; i < num_items * 2 && !terminar; i++) {
124         // 1. determinar si puede enviar solo prod., solo cons, o todos
125         int etiq;
126
127         if (num_celdas_ocupadas == 0) // si buffer vacío
128             // solo prods
129             etiq = etiq_prod;
130         else if (num_celdas_ocupadas == tam_vector) // si buffer lleno
131             // solo cons

```



```

128     etiq = etiq_cons;
129 else                                     // si no vacío ni lleno
130     // cualquiera
131     etiq = MPI_ANY_TAG;
132
133 // 2. recibir un mensaje del emisor o emisores aceptables
134
135 MPI_Recv(&valor, 1, MPI_INT, MPI_ANY_SOURCE, etiq, MPI_COMM_WORLD,
136         &estado); // Recibir mensaje del productor o consumidor
137
138 // 3. procesar el mensaje recibido
139
140 switch (estado.MPI_TAG) { // leer emisor del mensaje en metadatos
141
142     case etiq_prod: // si ha sido el productor: insertar en buffer
143         // Da permiso
144         MPI_Ssend(&valor, 1, MPI_INT, estado.MPI_SOURCE, 0,
145             MPI_COMM_WORLD); // Enviar confirmación al productor
146         MPI_Recv(&valor, 1, MPI_INT, estado.MPI_SOURCE, 0,
147             MPI_COMM_WORLD, &estado); // Recibir valor del productor
148
149         // Dos valores iguales consecutivos recibidos
150         if (valor == buffer[(primera_libre - 1 + tam_vector) %
151             tam_vector]) {
152             cout << "Empezamos a terminar el programa" << endl;
153             terminar = true;
154             int aux;
155             MPI_Request request;
156             // terminar procesos productores
157             for (int i = 0; i < id_buffer; i++) {
158                 MPI_Irecv(&aux, 1, MPI_INT, i, MPI_ANY_TAG,
159                     MPI_COMM_WORLD, &request); // Recibir cualquier
160                     mensaje pendiente
161                 MPI_Irecv(&aux, 1, MPI_INT, i, MPI_ANY_TAG,
162                     MPI_COMM_WORLD, &request); // Recibir cualquier
163                     mensaje pendiente
164                 aux = TERM;
165                 MPI_Ssend(&aux, 1, MPI_INT, i, 0, MPI_COMM_WORLD);
166                 // Enviar valor especial para finalizar
167             }
168
169             // terminar procesos consumidores
170             for (int i = id_buffer + 1; i < num_procesos_esperado;
171                 i++) {
172                 MPI_Irecv(&aux, 1, MPI_INT, i, MPI_ANY_TAG,
173                     MPI_COMM_WORLD, &request); // Recibir cualquier
174                     mensaje pendiente
175                 aux = TERM;
176                 MPI_Ssend(&aux, 1, MPI_INT, i, 0, MPI_COMM_WORLD);
177                 // Enviar valor especial para finalizar
178             }
179         }
180     }
181     if (!terminar) {
182         buffer[primera_libre] = valor; // Insertar valor en el
183             buffer
184         primera_libre = (primera_libre + 1) % tam_vector; //

```

```

171         Actualizar índice de primera celda libre
172         num_celdas_ocupadas++; // Incrementar número de celdas
173         ocupadas
174         cout << "Buffer ha recibido valor " << valor << endl;
175     }
176     break;
177
178     case etiq_cons: // si ha sido el consumidor: extraer y enviarle
179     if (!terminar) {
180         valor = buffer[primera_ocupada]; // Extraer valor del
181         buffer
182         primera_ocupada = (primera_ocupada + 1) % tam_vector;
183         // Actualizar índice de primera celda ocupada
184         num_celdas_ocupadas--; // Decrementar número de celdas
185         ocupadas
186         cout << "Buffer va a enviar valor " << valor << endl;
187         MPI_Ssend(&valor, 1, MPI_INT, estado.MPI_SOURCE, 0,
188             MPI_COMM_WORLD); // Enviar valor al consumidor
189     }
190     break;
191 }
192 cout << "Termina buffer" << endl; // Mensaje de finalización del buffer
193 }
194
195 // -----
196
197 int main(int argc, char *argv[]) {
198     int id_propio, num_procesos_actual;
199
200     if (num_items % num_prod != 0 || num_items % num_cons != 0) {
201         cout << "Numero de productores y consumidores no valido" << endl;
202         exit(1);
203     }
204
205     // inicializar MPI, leer identif. de proceso y número de procesos
206     MPI_Init(&argc, &argv);
207     MPI_Comm_rank(MPI_COMM_WORLD, &id_propio);
208     MPI_Comm_size(MPI_COMM_WORLD, &num_procesos_actual);
209
210     if (num_procesos_esperado == num_procesos_actual) {
211         // ejecutar la operación apropiada a 'id_propio'
212         if (id_ini_productores <= id_propio && id_propio <
213             id_ini_productores + num_prod)
214             funcion_productor(id_propio - id_ini_productores); // Ejecutar
215             función productor
216         else if (id_propio == id_buffer)
217             funcion_buffer(); // Ejecutar función buffer
218         else
219             funcion_consumidor(id_propio - id_ini_consumidores); //
220             Ejecutar función consumidor
221     } else {
222         if (id_propio == 0) // solo el primero escribe error, indep. del
223             rol
224         {
225             cout << "el número de procesos esperados es: "

```

```

217         << num_procesos_esperado << endl
218         << "el número de procesos en ejecución es: "
219         << num_procesos_actual << endl
220         << "(programa abortado)" << endl;
221     }
222 }
223
224 // al terminar el proceso, finalizar MPI
225 MPI_Finalize();
226 return 0;
227 }

```

Pincha aquí para acceder al fichero.

2 Examen de Mario Bros con Yoshi como NPC

2.1. Enunciado

Se pide resolver el siguiente ejercicio utilizando directivas MPI vistas en clase. El juego consiste en comer tantas tartas como sea posible. El que más kg de tartas haya comido, gana. Contaremos con las siguientes restricciones:

- Habrá solamente dos jugadores, Mario y Peach.
- Habrá un NPC, Yoshi, que será el árbitro.
- El proceso Yoshi se encargará:
 - Por un lado, de proveer tartas a cada equipo. Repondrá un máximo de x veces.
 - Cada vez que repone, decide si provee de 1 o 2 tartas a la vez. Creará un vector con 1 o 2 elementos.
 - Cada tarta tendrá un peso asociado que será también aleatorio entre 1 y 10 kg. Un vector [1,10] indicará el peso mínimo y máximo de las tartas.
 - Mostrará cuántas tartas va a reponer y a qué equipo.
 - Por otro lado, recibirá la suma de los kg que cada equipo se ha comido en cada tanda. Para cada tanda servida:
 - Comparará quién ha comido más kg de tarta en dicha tanda.
 - Sumará 1 punto al equipo que más kg haya comido.
 - Finalmente, comparará los puntos de ambos equipos y proclamará vencedor al que más kg de tarta comió.
- Los procesos Mario y Peach realizarán las siguientes operaciones:
 - Recibirán las tartas que se van a comer.
 - Mostrarán las tartas que han recibido y sus pesos.
 - Sumarán el peso de cada tarta y aleatoriamente se comerán una porción.

- La cantidad que se han comido se la notificarán a Yoshi.
- Mientras haya envíos que hacer, se han de poder realizar, evitando que haya ningún tipo de bloqueo. Es decir, Yoshi podrá reponer tartas sin tener que sufrir ningún tipo de bloqueo por otras operaciones (a excepción de retrasos aleatorios).
- Se han de incluir retrasos aleatorios.

2.2. Solución

```

1 //resolucion
2 #include <iostream>
3 #include <string>
4 #include <random>
5 #include <chrono>
6 #include <thread>
7 #include <mpi.h>
8 using namespace std ;
9
10 const int    ID_YOSHI = 0,
11             ID_MARIO = 1,
12             ID_PEACH = 2,
13             num_jugadores = 2,
14             num_procesos_esperados = num_jugadores + 1;
15
16 const double TERMINAR = -1;
17
18 const int    tag_jugador = 1,
19             tag_yoshi = 2;
20
21
22 /**
23  * @brief Función que produce un número aleatorio real entre dos valores.
24  *         El número aleatorio se genera en el rango [min, max].
25  *
26  * @param min Valor mínimo. Incluido.
27  * @param max Valor máximo. Incluido.
28  *
29  * @return double Número aleatorio generado.
30  */
31 double aleatorio_real(int min, int max) {
32     static const int precision = 1e3;
33     static default_random_engine generador( (random_device())() );
34     static uniform_int_distribution<int> distribucion_uniforme( 0,
35         precision ) ;
36     double valor_aleatorio = distribucion_uniforme( generador )/((double)
37         precision);
38     return min + valor_aleatorio * (max - min);
39 }
40
41 /**
42  * @brief Función que produce un número aleatorio entero entre dos valores.
43  *         El número aleatorio se genera en el rango [min, max].
44  *

```

```

43  * @param min Valor mínimo. Incluido.
44  * @param max Valor máximo. Incluido.
45  * @return int Número aleatorio generado.
46  */
47  int aleatorio_entero(int min, int max) {
48      return round(aleatorio_real(min, max));
49  }
50
51  /**
52   * @brief Función que bloquea el hilo actual durante un número de
53   *        milisegundos.
54   *
55   * @param num_milisegundos Número de milisegundos a bloquear.
56   */
57  void bloquear(int num_milisegundos) {
58      chrono::milliseconds duracion(num_milisegundos);
59      this_thread::sleep_for(duracion);
60  }
61
62  /**
63   * @brief Dado un id, devuelve el nombre del jugador/NPC correspondiente.
64   *
65   * @param id ID del jugador/NPC.
66   * @return string Nombre del jugador/NPC.
67   */
68  string id_a_nombre(int id) {
69      switch (id) {
70          case ID_YOSHI:
71              return "Yoshi";
72          case ID_MARIO:
73              return "Mario";
74          case ID_PEACH:
75              return "Peach";
76          default:
77              return "Desconocido";
78      }
79  }
80
81  /**
82   * @brief Función que ejecuta la hebra de Yoshi.
83   *
84   */
85  void funcion_yoshi() {
86      const int num_reposiciones = 10,
87              num_tartas_max_por_reposicion = 2,
88              peso_minimo_tarta = 1,
89              peso_maximo_tarta = 10;
90
91      int puntos[num_jugadores] = {0};
92
93      MPI_Status estado;
94
95      // Cada ronda
96      for (int i = 0; i < num_reposiciones; i++) {
97

```



```

98 // Enviamos las tartas a los jugadores
99 for (int id_jugador = 1; id_jugador <= num_jugadores; id_jugador++) {
100     // Número de tartas a enviar al jugador j
101     int num_tartas = aleatorio_entero(1, num_tartas_max_por_reposicion
102         );
103
104     // Pesos de las tartas a enviar
105     double * pesos_tartas = new double[num_tartas];
106     for (int k = 0; k < num_tartas; k++)
107         pesos_tartas[k] = aleatorio_real(peso_minimo_tarta,
108             peso_maximo_tarta);
109
110     // Producción de tartas
111     bloquear(aleatorio_entero(10, 200));
112     cout << "Yoshi: Número de tartas producidas para " << id_a_nombre(
113         id_jugador) << ": " << num_tartas << endl;
114
115     // Enviamos las tartas
116     MPI_Send(pesos_tartas, num_tartas, MPI_DOUBLE, id_jugador,
117         tag_yoshi, MPI_COMM_WORLD);
118
119     delete[] pesos_tartas;
120 }
121
122 // Esperamos a recibir los pesos comidos por los jugadores
123 double peso_comido_jugador;
124 double peso_comido_max = 0;
125 int id_ganador_ronda;
126 for (int j = 1; j <= num_jugadores; j++) {
127     MPI_Recv(&peso_comido_jugador, 1, MPI_DOUBLE, MPI_ANY_SOURCE,
128         tag_jugador, MPI_COMM_WORLD, &estado);
129     if (peso_comido_jugador > peso_comido_max) {
130         peso_comido_max = peso_comido_jugador;
131         id_ganador_ronda = estado.MPI_SOURCE;
132     }
133 }
134
135 // Sumamos un punto al jugador que más haya comido. Si hay empate, al
136 // que haya comido antes
137 puntos[id_ganador_ronda - 1]++;
138
139 // Informamos de los puntos actuales
140 cout << "Yoshi: Punto para " << id_a_nombre(id_ganador_ronda) << endl
141     ;
142 cout << "Yoshi: Puntos en la ronda " << i+1 << ": " << endl;
143 for (int j = 0; j < num_jugadores; j++)
144     cout << "\t- " << id_a_nombre(j+1) << ": " << puntos[j] << "
145         puntos." << endl;
146 cout << "-----" << endl;
147 }
148
149 // Finalizamos el juego
150 for (int j = 1; j <= num_jugadores; j++)
151     MPI_Send(&TERMINAR, 1, MPI_DOUBLE, j, tag_yoshi, MPI_COMM_WORLD);

```

```

146 // Informamos del ganador. Si hay empate, se queda con el que se
    apuntase antes
147 int id_ganador = 0;
148 for (int j = 0; j < num_jugadores; j++)
149     if (puntos[j] > puntos[id_ganador])
150         id_ganador = j;
151
152
153 cout << "Yoshi: ¡El ganador es " << id_a_nombre(id_ganador + 1) << "!"
    << endl;
154 }
155
156 /**
157  * @brief Función que ejecuta la hebra de Mario o Peach.
158  *
159  * @param id_jugador ID del jugador.
160  */
161 void funcion_player(int id_jugador) {
162
163     MPI_Status estado;
164     int num_tartas;
165     double peso_total, peso_comido;
166
167     while (true) {
168
169         // Esperamos a recibir el número de tartas
170         MPI_Probe(ID_YOSHI, tag_yoshi, MPI_COMM_WORLD, &estado);
171         MPI_Get_count(&estado, MPI_DOUBLE, &num_tartas);
172
173         // Una vez sabemos cuántas tartas hay, creamos un vector para
            almacenar sus pesos
174         double * pesos_tartas = new double[num_tartas];
175
176         // Recibimos las tartas
177         MPI_Recv(pesos_tartas, num_tartas, MPI_DOUBLE, estado.MPI_SOURCE,
            estado.MPI_TAG, MPI_COMM_WORLD, &estado);
178
179         // Comprobamos si se ha recibido el mensaje de terminar
180         if (num_tartas == 1 && pesos_tartas[0] == TERMINAR) {
181             cout << id_a_nombre(id_jugador) << ": ¡Termino!" << endl;
182             delete[] pesos_tartas;
183             break;
184         }
185
186         // Informamos de las tartas recibidas, y calculamos el peso total
187         peso_total = 0;
188         cout << id_a_nombre(id_jugador) << ": Número de tartas recibidas: "
            << num_tartas << endl;
189         for (int i = 0; i < num_tartas; i++) {
190             cout << "\t- Tarta " << i << ": " << pesos_tartas[i] << " kg." <<
                endl;
191             peso_total += pesos_tartas[i];
192         }
193
194         // Calculamos el peso que vamos a comer
195         peso_comido = aleatorio_real(0, peso_total);

```

```

196
197 // Simulamos acción de comer
198 cout << id_a_nombre(id_jugador) << ": Voy a comerme " << peso_comido
199 << " kg de tarta." << endl << endl;
200 bloquear(aleatorio_entero(10, 200));
201
202 // Informamos de que hemos terminado de comer
203 MPI_Send(&peso_comido, 1, MPI_DOUBLE, ID_YOSHI, tag_jugador,
204 MPI_COMM_WORLD);
205 delete[] pesos_tartas;
206 }
207 }
208
209 // -----
210 int main (int argc, char* argv[]) {
211     int id_propio, num_procesos_actual;
212
213     MPI_Init(&argc, &argv);
214     MPI_Comm_rank(MPI_COMM_WORLD, &id_propio);
215     MPI_Comm_size(MPI_COMM_WORLD, &num_procesos_actual);
216
217     if (num_procesos_actual != num_procesos_esperados) {
218         if (id_propio == 0) {
219             cerr << "El número de procesos esperados es " <<
220                 num_procesos_esperados << " y el número de procesos actual es "
221                 << num_procesos_actual << endl;
222         }
223         MPI_Finalize();
224         return 1;
225     }
226
227     if (id_propio == ID_YOSHI) { // Se trata de Yoshi
228         funcion_yoshi();
229     } else { // Se trata de un jugador
230         funcion_player(id_propio);
231     }
232
233     MPI_Finalize();
234     return 0;
235 }

```

3 Examen de Consumidores (valores pares e impares)

3.1. Enunciado

Se pide modificar la solución al problema de múltiples productores y consumidores realizada en clase utilizando directivas MPI, de forma que:

- El número de productores sea 3 (y que pueda variar fácilmente).
- El número de consumidores sea 2, de forma que:
 - Habrá un consumidor que solo pueda consumir los valores pares.

- El otro consumidor solo podrá consumir los valores impares.

3.2. Solución

3.2.1. Partimos de la plantilla

```

1 //
2 //
3 // Sistemas concurrentes y Distribuidos.
4 // Práctica 3. Implementación de algoritmos distribuidos con MPI
5 //
6 // Archivo: prodcons2.cpp
7 // Implementación del problema del productor-consumidor con
8 // un proceso intermedio que gestiona un buffer finito y recibe peticiones
9 // en orden arbitrario
10 // (versión con un único productor y un único consumidor)
11 //
12 // Historial:
13 // Actualizado a C++11 en Septiembre de 2017
14 //
15
16 #include <iostream>
17 #include <thread> // this_thread::sleep_for
18 #include <random> // dispositivos, generadores y distribuciones aleatorias
19 #include <chrono> // duraciones (duration), unidades de tiempo
20 #include <mpi.h>
21
22 using namespace std;
23 using namespace std::this_thread ;
24 using namespace std::chrono ;
25
26 const int
27     num_items          = 20,
28     tam_vector          = 10,
29     n_p = 4,
30     n_c = 5,
31     num_procesos_esperado = n_p + n_c + 1,
32     k = num_items/n_p;
33
34 //variables para el test de produccion de valores
35 int valores_producidos[n_p] = {0};
36 int valores_consumidos [n_c] = {0};
37
38 const int etiq_productor = 0, // etiqueta de productor
39          etiq_consumidor = 1; // etiqueta de consumidor
40
41
42 //*****
43 // plantilla de función para generar un entero aleatorio uniformemente
44 // distribuido entre dos valores enteros, ambos incluidos

```

```

45 // (ambos tienen que ser dos constantes, conocidas en tiempo de compilación
46 // -----
47
48 template< int min, int max > int aleatorio()
49 {
50     static default_random_engine generador( (random_device())() );
51     static uniform_int_distribution<int> distribucion_uniforme( min, max ) ;
52     return distribucion_uniforme( generador );
53 }
54 // -----
55 // producir produce los numeros en secuencia (1,2,3,...)
56 // y lleva espera aleatorio
57 int producir(int numero_productor) {
58     static int contador = 0;
59     sleep_for(milliseconds(aleatorio<10, 100>()));
60     contador++;
61     cout << "Productor " << numero_productor << " ha producido valor " <<
        contador << endl << flush;
62     valores_producidos[numero_productor]++;
63     return contador;
64 }
65 // -----
66
67 void funcion_productor(int numero_productor) {
68     for (int i = 0; i < k; i++) {
69         int valor_prod = producir(numero_productor);
70         MPI_Ssend(&valor_prod, 1, MPI_INT, n_p, etiq_productor,
            MPI_COMM_WORLD);
71         cout << "Productor " << numero_productor << " envió valor " <<
            valor_prod << endl << flush;
72     }
73 }
74 // -----
75
76 void consumir( int valor_cons )
77 {
78     // espera bloqueada
79     sleep_for( milliseconds( aleatorio<110,200>() ) );
80     cout << "Consumidor ha consumido valor " << valor_cons << endl << flush
81         ;
82 }
83 // -----
84 void funcion_consumidor(int numero_consumidor)
85 {
86     int          peticion = 1,
87               valor_rec;
88     MPI_Status  estado ;
89
90     for( unsigned int i=0 ; i < num_items/n_c; i++ )
91     {
92         MPI_Ssend( &peticion, 1, MPI_INT, n_p, etiq_consumidor,
93             MPI_COMM_WORLD); //n_p hace referencia al id de buffer
94         MPI_Recv ( &valor_rec, 1, MPI_INT, n_p, etiq_consumidor,
95             MPI_COMM_WORLD,&estado );

```



```

94     cout << "Consumidor con rol " << numero_consumidor << " ha recibido
95         valor " << valor_rec << endl << flush ;
96     consumir( valor_rec );
97     valores_consumidos[numero_consumidor]++;
98 }
99 // -----
100
101 void funcion_buffer()
102 {
103     int          buffer[tam_vector],      // buffer con celdas ocupadas y vacías
104         as
105         valor,                          // valor recibido o enviado
106         primera_libre    = 0, // índice de primera celda libre
107         primera_ocupada  = 0, // índice de primera celda ocupada
108         num_celdas_ocupadas = 0; // número de celdas ocupadas
109     MPI_Status estado ;                  // metadatos del mensaje recibido
110
111     int etiq_aceptable; // etiqueta aceptable, almacena el valor de la
112                          // etiqueta del mensaje recibido
113
114     for( unsigned int i=0 ; i < num_items*2 ; i++ )
115     {
116         etiq_aceptable = (num_celdas_ocupadas == 0) ? etiq_productor
117                     : (num_celdas_ocupadas == tam_vector) ?
118                         etiq_consumidor
119                     : MPI_ANY_TAG;
120
121         // 2. recibir un mensaje del emisor o emisores aceptables
122
123         MPI_Recv( &valor, 1, MPI_INT, MPI_ANY_SOURCE, etiq_aceptable,
124                 MPI_COMM_WORLD, &estado );
125
126         // 3. procesar el mensaje recibido
127
128         switch( estado.MPI_TAG ) // leer emisor del mensaje en metadatos
129         {
130             case etiq_productor: // si ha sido el productor: insertar en
131                 buffer
132                 buffer[primera_libre] = valor ;
133                 primera_libre = (primera_libre+1) % tam_vector ;
134                 num_celdas_ocupadas++ ;
135                 cout << "Buffer ha recibido valor " << valor << endl ;
136                 break;
137
138             case etiq_consumidor: // si ha sido el consumidor: extraer y
139                 enviarle
140                 valor = buffer[primera_ocupada] ;
141                 primera_ocupada = (primera_ocupada+1) % tam_vector ;
142                 num_celdas_ocupadas-- ;
143                 cout << "Buffer va a enviar valor " << valor << endl ;
144                 MPI_Ssend( &valor, 1, MPI_INT, estado.MPI_SOURCE ,
145                         etiq_consumidor, MPI_COMM_WORLD);
146                 break;
147         }
148     }
149 }

```

```

142 }
143
144 //Función extra para verificar que cada productor ha producido la cantidad
    correcta de ítems
145 void test_produccion() {
146
147     for (int i = 0; i < n_p; i++)
148     {
149         if(valores_producidos[i] == k){
150             cout << endl << "
                -----" <<
151                 endl;
                cout << "| Productor " << i << " ha producido la cantidad correcta
                    de ítems |" << endl;
152             cout << "
                -----" <<
                endl;
153         }
154     }
155
156     for (int i = 0; i < n_c; i++)
157     {
158         if (valores_consumidos[i] == num_items/n_c)
159         {
160             cout << endl << "
                -----" <<
161                 endl;
                cout << "| Consumidor " << i << " ha consumido la cantidad
                    correcta de ítems |" << endl;
162             cout << "
                -----" <<
                endl;
163         }
164     }
165
166
167
168
169 }
170
171
172 // -----
173 int main( int argc, char *argv[] )
174 {
175     int id_propio, num_procesos_actual;
176
177     // inicializar MPI, leer identif. de proceso y número de procesos
178     MPI_Init( &argc, &argv );
179     MPI_Comm_rank( MPI_COMM_WORLD, &id_propio );
180     MPI_Comm_size( MPI_COMM_WORLD, &num_procesos_actual );
181
182     if(num_procesos_esperado == num_procesos_actual) {
183         if(id_propio < n_p) {
184             funcion_productor(id_propio);
185
186         }
    }

```

```

187     else if (id_propio == n_p) {
188         funcion_buffer();
189     }
190     else {
191         funcion_consumidor(id_propio - n_p - 1); // debemos de restarlo
192             para que coincida con el numero del proceso consumidor
193     }
194 } else {
195     if ( id_propio == 0 ) // solo el primero escribe error, indep. del
196         rol
197     { cout << "el número de procesos esperados es: " <<
198         num_procesos_esperado << endl
199         << "el número de procesos en ejecución es: " <<
200         num_procesos_actual << endl
201         << "(programa abortado)" << endl ;
202     }
203 }
204
205 // al terminar el proceso, finalizar MPI
206 MPI_Finalize( );
207
208 test_produccion();
209 return 0;
210 }

```

3.2.2. Solución

```

1 //
2 // -----
3 //
4 // Sistemas concurrentes y Distribuidos.
5 // Práctica 3. Implementación de algoritmos distribuidos con MPI
6 //
7 // Archivo: prodcons2.cpp
8 // Implementación del problema del productor-consumidor con
9 // un proceso intermedio que gestiona un buffer finito y recibe peticiones
10 // en orden arbitrario
11 // (versión con un único productor y un único consumidor)
12 //
13 // Historial:
14 // Actualizado a C++11 en Septiembre de 2017
15 // -----
16
17 #include <iostream>
18 #include <thread> // this_thread::sleep_for
19 #include <random> // dispositivos, generadores y distribuciones aleatorias
20 #include <chrono> // duraciones (duration), unidades de tiempo
21 #include <mpi.h>
22
23 using namespace std;

```

```

23 using namespace std::this_thread ;
24 using namespace std::chrono ;
25
26 const int
27     num_items          = 6, //cambiamos los datos para que sean
        divisibles entre productor y consumidor
28     tam_vector          = 10,
29     n_p = 3,
30     n_c = 2,
31     num_procesos_esperado = n_p + n_c + 1,
32     k = num_items/n_p;
33
34 //variables para el test de produccion de valores
35 int valores_producidos[n_p] = {0};
36 int valores_consumidos [n_c] = {0};
37
38 const int etiq_productor = 0, // etiqueta de productor
39         //etiq_consumidor = 1; // etiqueta de consumidor
40     etiq_consumidor_par = 1, // etiqueta de consumidor par
41     etiq_consumidor_impar = 2, // etiqueta de consumidor impar
42     etiq_consumidor = 3; // etiqueta de consumidor
43
44
45 //*****
46 // plantilla de función para generar un entero aleatorio uniformemente
47 // distribuido entre dos valores enteros, ambos incluidos
48 // (ambos tienen que ser dos constantes, conocidas en tiempo de compilación
49 // )
50 //-----
51 template< int min, int max > int aleatorio()
52 {
53     static default_random_engine generador( (random_device())() );
54     static uniform_int_distribution<int> distribucion_uniforme( min, max ) ;
55     return distribucion_uniforme( generador );
56 }
57 // -----
58 // producir produce los numeros en secuencia (1,2,3,...)
59 // y lleva espera aleatorio
60 int producir(int numero_productor) {
61     static int contador = 0;
62     sleep_for(milliseconds(aleatorio<10, 100>()));
63     contador++;
64     cout << "Productor " << numero_productor << " ha producido valor " <<
        contador << endl << flush;
65     valores_producidos[numero_productor]++;
66     return contador;
67 }
68 // -----
69
70 void funcion_productor(int numero_productor) {
71     for (int i = 0; i < k; i++) {
72         int valor_prod = producir(numero_productor);
73         MPI_Ssend(&valor_prod, 1, MPI_INT, n_p, etiq_productor,
            MPI_COMM_WORLD);

```

```

74     cout << "Productor " << numero_productor << " envió valor " <<
       valor_prod << endl << flush;
75 }
76 }
77 // -----
78
79 void consumir( int valor_cons )
80 {
81     // espera bloqueada
82     sleep_for( milliseconds( aleatorio<110,200>()) );
83     cout << "Consumidor ha consumido valor " << valor_cons << endl << flush
84         ;
85 }
86 // -----
87 void funcion_consumidor(int numero_consumidor)
88 {
89     int          peticion = 1,
90               valor_rec;
91     MPI_Status  estado ;
92
93     for( unsigned int i=0 ; i < num_items/n_c; i++)
94     {
95         MPI_Ssend( &peticion, 1, MPI_INT, n_p, etiq_consumidor,
96                   MPI_COMM_WORLD); //n_p hace referencia al id de buffer
97
98         MPI_Recv ( &valor_rec, 1, MPI_INT, n_p, MPI_ANY_TAG, MPI_COMM_WORLD,&
99                   estado );
100         if( estado.MPI_TAG == etiq_consumidor_par ){
101             cout << "Consumidor con rol " << numero_consumidor << " ha
102                 recibido valor " << valor_rec << " que es par" << endl << flush
103                 ;
104         }
105         else if( estado.MPI_TAG == etiq_consumidor_impar ){
106             cout << "Consumidor con rol " << numero_consumidor << " ha
107                 recibido valor " << valor_rec << " que es impar" << endl <<
108                 flush ;
109         }
110         else{
111             cout << "ERROR, HA RECIBIDO DEL PRODUCTOR, .... ABORTAR....";
112             exit(0);
113         }
114         //cout << "Consumidor con rol " << numero_consumidor << " ha recibido
115         //valor " << valor_rec << endl << flush ;
116         consumir( valor_rec );
117         valores_consumidos[numero_consumidor]++;
118     }
119 }
120 // -----
121
122 void funcion_buffer()
123 {
124     int          buffer[tam_vector],          // buffer con celdas ocupadas y vací
125           as
126           valor,                               // valor recibido o enviado
127           primera_libre = 0, // índice de primera celda libre

```

```

120         primera_ocupada      = 0, // índice de primera celda ocupada
121         num_celdas_ocupadas = 0; // número de celdas ocupadas
122     MPI_Status estado ;           // metadatos del mensaje recibido
123
124     int etiq_aceptable; // etiqueta aceptable, almacena el valor de la
125                          // etiqueta del mensaje recibido
126
127     for( unsigned int i=0 ; i < num_items*2 ; i++ )
128     {
129         etiq_aceptable = (num_celdas_ocupadas == 0) ? etiq_productor
130                     : (num_celdas_ocupadas == tam_vector) ?
131                         etiq_consumidor
132                     : MPI_ANY_TAG;
133
134         // 2. recibir un mensaje del emisor o emisores aceptables
135
136         MPI_Recv( &valor, 1, MPI_INT, MPI_ANY_SOURCE, etiq_aceptable,
137                 MPI_COMM_WORLD, &estado );
138
139         // 3. procesar el mensaje recibido
140
141         switch( estado.MPI_TAG ) // leer emisor del mensaje en metadatos
142         {
143             case etiq_productor: // si ha sido el productor: insertar en
144                 buffer
145                 buffer[primera_libre] = valor ;
146                 primera_libre = (primera_libre+1) % tam_vector ;
147                 num_celdas_ocupadas++ ;
148                 cout << "Buffer ha recibido valor " << valor << endl ;
149                 break;
150
151             case etiq_consumidor: // si ha sido el consumidor: extraer y
152                 enviarle
153                 valor = buffer[primera_ocupada] ;
154                 primera_ocupada = (primera_ocupada+1) % tam_vector ;
155                 num_celdas_ocupadas-- ;
156                 cout << "Buffer va a enviar valor " << valor << endl ;
157                 if(valor%2 == 0){
158                     cout << "Buffer va a enviar valor " << valor << " a
159                         consumidor par" << endl ;
160                     MPI_Ssend( &valor, 1, MPI_INT, estado.MPI_SOURCE ,
161                             etiq_consumidor_par, MPI_COMM_WORLD);
162                 }
163                 else{
164                     cout << "Buffer va a enviar valor " << valor << " a
165                         consumidor impar" << endl ;
166                     MPI_Ssend( &valor, 1, MPI_INT, estado.MPI_SOURCE ,
167                             etiq_consumidor_impar, MPI_COMM_WORLD);
168                 }
169                 break;
170         }
171     }
172 }

```



```

166 //Función extra para verificar que cada productor ha producido la cantidad
    correcta de ítems
167 void test_produccion() {
168
169     for (int i = 0; i < n_p; i++)
170     {
171         if(valores_producidos[i] == k){
172             cout << endl << "
                -----" <<
                endl;
173             cout << "| Productor " << i << " ha producido la cantidad correcta
                de ítems |" << endl;
174             cout << "
                -----" <<
                endl;
175         }
176     }
177
178     for (int i = 0; i < n_c; i++)
179     {
180         if (valores_consumidos[i] == num_items/n_c)
181         {
182             cout << endl << "
                -----" <<
                endl;
183             cout << "| Consumidor " << i << " ha consumido la cantidad
                correcta de ítems |" << endl;
184             cout << "
                -----" <<
                endl;
185         }
186     }
187
188
189
190
191 }
192
193
194 // -----
195 int main( int argc, char *argv[] )
196 {
197     int id_propio, num_procesos_actual;
198
199     // inicializar MPI, leer identif. de proceso y número de procesos
200     MPI_Init( &argc, &argv );
201     MPI_Comm_rank( MPI_COMM_WORLD, &id_propio );
202     MPI_Comm_size( MPI_COMM_WORLD, &num_procesos_actual );
203
204     if(num_procesos_esperado == num_procesos_actual) {
205         if(id_propio < n_p) {
206             funcion_productor(id_propio);
207
208         }
209         else if (id_propio == n_p) {
210             funcion_buffer();

```

```

211     }
212     else {
213         funcion_consumidor(id_propio - n_p - 1); // debemos de restarlo
214             para que coincida con el numero del proceso consumidor
215     }
216 } else {
217     if ( id_propio == 0 ) // solo el primero escribe error, indep. del
218         rol
219     { cout << "el número de procesos esperados es: " <<
220         num_procesos_esperado << endl
221         << "el número de procesos en ejecución es: " <<
222         num_procesos_actual << endl
223         << "(programa abortado)" << endl ;
224     }
225 }
226
227 // al terminar el proceso, finalizar MPI
228 MPI_Finalize( );
229
230 test_produccion();
231 return 0;
232 }

```

4 Examen de modificar camarero y filósofos

4.1. Enunciado

El archivo plantilla proporcionado contiene una solución básica al problema de los filósofos con camarero. En esta solución, el camarero se encarga de controlar el acceso a la mesa, permitiendo que se sienten a la mesa un máximo de 4 filósofos. Hay 5 filósofos.

Modificar este archivo como sigue:

- El filósofo 4, cuyo rango es 8, tendrá un comportamiento algo diferente del resto. El filósofo 4 no pide ni suelta ningún tenedor ya que come con las manos. No obstante, este filósofo sí debe pedir permiso para sentarse y levantarse al camarero.
- Como el filósofo 4 no puede comer si no se siente muy acompañado, el camarero solo permite que se siente cuando hay 4 filósofos sentados en ella.
- El camarero además no permite levantarse a ninguno de ellos mientras está comiendo el filósofo 4, para que se sienta acompañado.
- Como ahora ya no hay interbloqueo, podría haber 5 sentados en la mesa. Han de usarse unas etiquetas especiales para el filósofo 4.

El camarero deberá, en un bucle infinito:

- Paso 1 (Usar Iprobe)

- IF (hay menos de 4 sentados)
 - Sondeo en un bucle de las peticiones de sentarse o levantarse de cualquier filósofo distinto al 4, hasta detectar una.
 - Recibe el primer mensaje detectado.
 - ELSE IF (hay 4)
 - Sondeo en un bucle de las peticiones de levantarse de los filósofos distintos al 4 y de sentarse del 4, hasta detectar una.
 - Recibe el primer mensaje detectado.
 - ELSE
 - Recibe la petición de levantarse del filósofo 4.
- Paso 2
- Actualiza el número de sentados (en función de la etiqueta recibida).

4.2. Solución

4.2.1. Partimos de la plantilla

```

1
2 #include <mpi.h>
3 #include <thread> // this_thread::sleep_for
4 #include <random> // dispositivos, generadores y distribuciones aleatorias
5 #include <chrono> // duraciones (duration), unidades de tiempo
6 #include <iostream>
7
8 using namespace std;
9 using namespace std::this_thread ;
10 using namespace std::chrono ;
11
12 const int
13     num_filosofos = 5 ,           // número de filósofos
14     num_filo_ten  = 2*num_filosofos, // número de filósofos y tenedores
15     num_procesos  = num_filo_ten+1 ; // número de procesos total
16 const int camarero= num_filo_ten;
17 const int tag_sentar=0, tag_levantar=1;
18
19
20 //*****
21 // plantilla de función para generar un entero aleatorio uniformemente
22 //-----
23 template< int min, int max > int aleatorio()
24 {
25     static default_random_engine generador( (random_device())() );
26     static uniform_int_distribution<int> distribucion_uniforme( min, max ) ;
27     return distribucion_uniforme( generador );
28 }
29
30 // -----
31
32 void funcion_filosofos( int id )

```

```

33 {
34     int id_ten_izq = (id+1)                % num_filo_ten, //id. tenedor izq.
35     id_ten_der = (id+num_filo_ten-1) % num_filo_ten; //id. tenedor der.
36     int peticion=0, valor;
37
38     while ( true )
39     {
40
41         cout <<"+++++ Filosofo " <<id << " solicita
42             SENTARSE." <<endl;
43         MPI_Ssend(&peticion, 1, MPI_INT, camarero, tag_sentar, MPI_COMM_WORLD);
44
45         cout <<"Filosofo " <<id << " solicita ten. izq." <<id_ten_izq <<endl;
46         MPI_Ssend(&peticion, 1, MPI_INT, id_ten_izq, 0, MPI_COMM_WORLD);
47
48         cout <<"Filosofo " <<id <<" solicita ten. der." <<id_ten_der <<endl;
49         MPI_Ssend(&peticion, 1, MPI_INT, id_ten_der, 0, MPI_COMM_WORLD);
50
51         cout <<endl<<"Filosofo " <<id <<" comienza a COMER" <<endl ;
52         sleep_for( milliseconds( aleatorio<1000,1800>() ) );
53
54         cout <<"Filosofo " <<id <<" suelta ten. izq. " <<id_ten_izq <<endl;
55         MPI_Ssend(&peticion, 1, MPI_INT, id_ten_izq, 0, MPI_COMM_WORLD);
56
57         cout<< "Filosofo " <<id <<" suelta ten. der. " <<id_ten_der <<endl;
58         MPI_Ssend(&peticion, 1, MPI_INT, id_ten_der, 0, MPI_COMM_WORLD);
59
60         cout <<"+++++ Filosofo " <<id << " solicita
61             LEVANTARSE." <<endl;
62         MPI_Ssend(&peticion, 1, MPI_INT, camarero, tag_levantar, MPI_COMM_WORLD);
63
64         cout<<endl << "Filosofo " << id << " comienza a PENSAR" << endl;
65         sleep_for( milliseconds( aleatorio<1000,1800>() ) );
66     }
67 }
68
69
70 // -----
71 void funcion_tenedores( int id )
72 {
73     int valor, id_filosofo ; // valor recibido, identificador del filósofo
74     MPI_Status estado ;      // metadatos de las dos recepciones
75
76     while ( true )
77     {
78         MPI_Recv(&valor, 1, MPI_INT, MPI_ANY_SOURCE, 0, MPI_COMM_WORLD, &estado);
79         id_filosofo=estado.MPI_SOURCE;
80         cout <<".....Ten. " <<id <<
81             " ha sido COGIDO por filo. " <<id_filosofo <<endl;
82
83         MPI_Recv(&valor, 1, MPI_INT, id_filosofo, 0, MPI_COMM_WORLD, &estado);
84         cout <<".....Ten. " << id <<
85             " ha sido SOLTADO por filo. " <<id_filosofo <<endl
86             ;

```

```

86     }
87 }
88
89 // -----
90 void funcion_camarero( )
91 {
92     int sentados=0, id_filosofo, tag, valor;
93     MPI_Status estado ;
94
95     while ( true )
96     {
97         if (sentados<4)
98             MPI_Recv(&valor, 1, MPI_INT, MPI_ANY_SOURCE, MPI_ANY_TAG,
99                     MPI_COMM_WORLD, &estado);
100         else
101             MPI_Recv(&valor, 1, MPI_INT, MPI_ANY_SOURCE, tag_levantar,
102                     MPI_COMM_WORLD, &estado);
103
104         id_filosofo=estado.MPI_SOURCE; tag=estado.MPI_TAG;
105
106         if (tag==tag_sentar)
107             sentados++;
108         else
109             sentados--;
110         cout <<endl<<"#####
111             CAMARERO DICE QUE HAY " <<sentados <<
112             " FILOSOFOS EN LA MESA " <<endl
113             <<endl;
114     }
115 }
116 // -----
117
118 int main( int argc, char** argv )
119 {
120     int id_propio, num_procesos_actual ;
121
122     MPI_Init( &argc, &argv );
123     MPI_Comm_rank( MPI_COMM_WORLD, &id_propio );
124     MPI_Comm_size( MPI_COMM_WORLD, &num_procesos_actual );
125
126     if ( num_procesos == num_procesos_actual )
127     {
128         if (id_propio==camarero)
129             funcion_camarero();
130         else if ( id_propio % 2 == 0 )
131             funcion_filosofos( id_propio );
132         else
133             funcion_tenedores( id_propio );
134     }
135     else
136     {
137         if ( id_propio == 0 ) // solo el primero escribe error, indep. del
138                               rol

```

```

136     { cout << "el numero de procesos debería ser:  " << num_procesos <<
        endl
137         << "    y el numero de procesos es: " << num_procesos_actual <<
            endl
138         << "(programa abortado)" << endl ;
139     }
140 }
141
142 MPI_Finalize( );
143 return 0;
144 }
145
146 // -----

```

4.2.2. Solución

```

1
2 #include <mpi.h>
3 #include <thread> // this_thread::sleep_for
4 #include <random> // dispositivos, generadores y distribuciones aleatorias
5 #include <chrono> // duraciones (duration), unidades de tiempo
6 #include <iostream>
7
8 using namespace std;
9 using namespace std::this_thread ;
10 using namespace std::chrono ;
11
12 const int
13     num_filosofos = 5 ,           // número de filósofos
14     num_filo_ten  = 2*num_filosofos, // número de filósofos y tenedores
15     num_procesos  = num_filo_ten+1 ; // número de procesos total
16 const int camarero= num_filo_ten;
17 const int tag_sentar=0, tag_levantar=1;
18 const int fil4 = 8;
19 const int tag_sentarsef4=2, tag_levantarsef4=3;
20
21
22 //*****
23 // plantilla de función para generar un entero aleatorio uniformemente
24 //-----
25 template< int min, int max > int aleatorio()
26 {
27     static default_random_engine generador( (random_device())() );
28     static uniform_int_distribution<int> distribucion_uniforme( min, max ) ;
29     return distribucion_uniforme( generador );
30 }
31
32 // -----
33
34 void funcion_filosofos( int id )
35 {
36     int id_ten_izq = (id+1)           % num_filo_ten, //id. tenedor izq.
37     id_ten_der  = (id+num_filo_ten-1) % num_filo_ten; //id. tenedor der.
38     int peticion=0, valor;

```



```

39 while ( true )
40 {
41
42
43     if(id!=fil4){
44         cout <<"+++++ Filosofo " <<id << " solicita
45             SENTARSE." <<endl;
46         MPI_Ssend(&peticion, 1, MPI_INT, camarero, tag_sentar, MPI_COMM_WORLD);
47
48         cout <<"Filosofo " <<id << " solicita ten. izq." <<id_ten_izq <<endl;
49         MPI_Ssend(&peticion, 1, MPI_INT, id_ten_izq, 0, MPI_COMM_WORLD);
50
51         cout <<"Filosofo " <<id << " solicita ten. der." <<id_ten_der <<endl;
52         MPI_Ssend(&peticion, 1, MPI_INT, id_ten_der, 0, MPI_COMM_WORLD);
53
54         cout <<endl<<"Filosofo " <<id << " comienza a COMER" <<endl ;
55         sleep_for( milliseconds( aleatorio<1000,1800>() ) );
56
57         cout <<"Filosofo " <<id << " suelta ten. izq. " <<id_ten_izq <<endl;
58         MPI_Ssend(&peticion, 1, MPI_INT, id_ten_izq, 0, MPI_COMM_WORLD);
59
60         cout<< "Filosofo " <<id << " suelta ten. der. " <<id_ten_der <<endl;
61         MPI_Ssend(&peticion, 1, MPI_INT, id_ten_der, 0, MPI_COMM_WORLD);
62
63         cout <<"+++++ Filosofo " <<id << " solicita
64             LEVANTARSE." <<endl;
65         MPI_Ssend(&peticion, 1, MPI_INT, camarero, tag_levantar, MPI_COMM_WORLD)
66             ;
67
68         cout<<endl << "Filosofo " << id << " comienza a PENSAR" << endl;
69         sleep_for( milliseconds( aleatorio<1000,1800>() ) );
70     }
71     else if (id == fil4){
72         //si es el filósofo 4, no pide ni suelta ningún tenedor, ya que come
73         //con las manos
74         cout <<"+++++ Filosofo ESPECIAL" <<id << "
75             solicita SENTARSE." <<endl;
76         MPI_Ssend(&peticion, 1, MPI_INT, camarero, tag_sentarsef4,
77             MPI_COMM_WORLD); //usamos una etiqueta especial para que la
78             reconozca el camarero
79
80         cout <<endl<<"Filosofo ESPECIAL" <<id << " comienza a COMER" <<endl ;
81         sleep_for( milliseconds( aleatorio<1000,1800>() ) );
82
83         cout <<"+++++ Filosofo ESPECIAL" <<id << "
84             solicita LEVANTARSE." <<endl;
85         MPI_Ssend(&peticion, 1, MPI_INT, camarero, tag_levantarsef4,
86             MPI_COMM_WORLD); //usamos una etiqueta especial para que la
87             reconozca el camarero
88
89         cout<<endl << "Filosofo ESPECIAL" << id << " comienza a PENSAR" <<
90             endl;
91         sleep_for( milliseconds( aleatorio<1000,1800>() ) );
92     }
93 }

```

```

84     }
85 }
86 }
87 }
88
89
90 // -----
91 void funcion_tenedores( int id )
92 {
93     int valor, id_filosofo ; // valor recibido, identificador del filósofo
94     MPI_Status estado ;      // metadatos de las dos recepciones
95
96     while ( true )
97     {
98         MPI_Recv(&valor, 1, MPI_INT, MPI_ANY_SOURCE, 0, MPI_COMM_WORLD, &estado);
99         id_filosofo=estado.MPI_SOURCE;
100         cout <<".....Ten. " <<id <<
101             " ha sido COGIDO por filo. " <<id_filosofo <<endl;
102
103         MPI_Recv(&valor, 1, MPI_INT, id_filosofo, 0, MPI_COMM_WORLD, &estado);
104         cout <<".....Ten. " << id <<
105             " ha sido SOLTADO por filo. " <<id_filosofo <<endl
106             ;
107     }
108 }
109 // -----
110 void funcion_camarero( )
111 {
112     int sentados = 0, id_filosofo, tag, valor;
113     MPI_Status estado;
114
115     while ( true )
116     {
117         if (sentados < 4) // si hay menos de 4 filósofos sentados
118         {
119             int mensaje_recibido = 0;
120             while (!mensaje_recibido)
121             {
122                 MPI_Iprobe(MPI_ANY_SOURCE, MPI_ANY_TAG, MPI_COMM_WORLD, &
123                     mensaje_recibido, &estado);
124                 if (mensaje_recibido)
125                 {
126                     MPI_Recv(&valor, 1, MPI_INT, estado.MPI_SOURCE, estado.MPI_TAG,
127                         MPI_COMM_WORLD, &estado);
128                 }
129             }
130         }
131         else if (sentados == 4) // si hay 4 filósofos sentados
132         {
133             int mensaje_recibido = 0;
134             while (!mensaje_recibido)
135             {
136                 MPI_Iprobe(MPI_ANY_SOURCE, MPI_ANY_TAG, MPI_COMM_WORLD, &
137                     mensaje_recibido, &estado);

```

```

135     if (mensaje_recibido && (estado.MPI_TAG == tag_levantar || (estado.
136         MPI_TAG == tag_sentarsef4 && estado.MPI_SOURCE == fil4)))
137     {
138         MPI_Recv(&valor, 1, MPI_INT, estado.MPI_SOURCE, estado.MPI_TAG,
139             MPI_COMM_WORLD, &estado);
140     }
141 }
142 else // si hay más de 4 filósofos sentados (solo puede ser el filósofo
143     4)
144 {
145     MPI_Recv(&valor, 1, MPI_INT, fil4, tag_levantarsef4, MPI_COMM_WORLD,
146         &estado);
147 }
148
149 id_filosofo = estado.MPI_SOURCE;
150 tag = estado.MPI_TAG;
151
152 if (tag == tag_sentar || tag == tag_sentarsef4)
153     sentados++;
154 else if (tag == tag_levantar || (tag == tag_levantarsef4))
155     sentados--;
156
157 cout << endl << "#####
158     CAMARERO DICE QUE HAY " << sentados <<
159     " FILOSOFOS EN LA MESA " << endl << endl;
160 }
161 }
162 // -----
163
164 int main( int argc, char** argv )
165 {
166     int id_propio, num_procesos_actual ;
167
168     MPI_Init( &argc, &argv );
169     MPI_Comm_rank( MPI_COMM_WORLD, &id_propio );
170     MPI_Comm_size( MPI_COMM_WORLD, &num_procesos_actual );
171
172     if ( num_procesos == num_procesos_actual )
173     {
174         if (id_propio==camarero)
175             funcion_camarero();
176         else if ( id_propio % 2 == 0 )
177             funcion_filosofos( id_propio );
178         else
179             funcion_tenedores( id_propio );
180     }
181     else
182     {
183         if ( id_propio == 0 ) // solo el primero escribe error, indep. del
184             rol
185         { cout << "el numero de procesos debería ser: " << num_procesos <<
186             endl
187             << " y el numero de procesos es: " << num_procesos_actual <<
188             endl

```

```

183         << "(programa abortado)" << endl ;
184     }
185 }
186
187 MPI_Finalize( );
188 return 0;
189 }
190
191 // -----

```

5 Examen Realizado del grupo A3 de este curso

5.1. Enunciado

Se iba detallando los pasos que se debía de hacer, en este caso en la función del proceso encargado se debía de consultar el valor de cajas disponibles, en mi caso lo hice así por simplicidad.

5.2. Solución

```

1 //Ismael Sallami Moreno
2 //Doble Grado en Ingeniería Informática + ADE
3
4 //para la resolución de este problema vamos a usar la plantilla del
5 //problema de los filósofos con un camarero
6
7 #include <mpi.h>
8 #include <thread> // this_thread::sleep_for
9 #include <random> // dispositivos, generadores y distribuciones aleatorias
10 #include <chrono> // duraciones (duration), unidades de tiempo
11 #include <iostream>
12
13 using namespace std;
14 using namespace std::this_thread ;
15 using namespace std::chrono ;
16
17 const int
18     cajas = 3, // número de cajas registradoras
19     clientes = 10, // número de clientes
20     num_procesos = 11, // número de procesos
21     id_proceso_encargado = num_procesos-1; // id del proceso encargado
22
23 int peticion_pago = 1; //para solicitar pagar
24 int peticion_fin_pago=2; //para solicitar el fin de pago
25
26 bool caja_vacia[cajas] = {true,true,true};
27
28 int num_caja_vacia(){
29     int val = 0;
30     for(int i=0;i<cajas;i++){
31         if(caja_vacia[i]){

```

```

32     }
33 }
34 return val;
35 }
36 int caja_libre(){
37     int id_caja_libre=-1;
38     for(int i=0;i<cajas;i++){
39         if(caja_vacia[i]){
40             id_caja_libre = i;
41             break;
42         }
43     }
44     return id_caja_libre;
45 }
46
47 //*****
48 // plantilla de función para generar un entero aleatorio uniformemente
49 // distribuido entre dos valores enteros, ambos incluidos
50 // (ambos tienen que ser dos constantes, conocidas en tiempo de compilación
51 // )
52 //-----
53 template< int min, int max > int aleatorio()
54 {
55     static default_random_engine generador( (random_device())() );
56     static uniform_int_distribution<int> distribucion_uniforme( min, max )
57     ;
58     return distribucion_uniforme( generador );
59 }
60 // -----
61
62 void funcion_cliente(int id){
63     MPI_Status estado;
64     int caja_donde_pagar; // valor que recibe del proceso encargado
65     int valor = id; // enviar el ID del cliente al encargado
66
67     while(true) {
68         sleep_for( milliseconds( aleatorio<10,300>() ) ); // espera
69         aleatoria
70         cout << "Cliente " << id << ": solicito que se me asigne caja" <<
71         endl;
72         MPI_Ssend(&valor, 1, MPI_INT, id_proceso_encargado, peticion_pago,
73         MPI_COMM_WORLD);
74         MPI_Recv(&caja_donde_pagar, 1, MPI_INT, id_proceso_encargado, 0,
75         MPI_COMM_WORLD, &estado);
76         cout << "Cliente " << id << ": comienzo pago en caja " <<
77         caja_donde_pagar << endl;
78         sleep_for( milliseconds( aleatorio<10,300>() ) ); // espera
79         aleatoria
80         cout << "Cliente " << id << ": termino pago en caja " <<
81         caja_donde_pagar << endl;
82         MPI_Ssend(&caja_donde_pagar, 1, MPI_INT, id_proceso_encargado,
83         peticion_fin_pago, MPI_COMM_WORLD);
84     }
85 }

```

```

78
79
80 void funcion_encargado(){
81     int valor, caja_a_liberar; // valor es el id del cliente o la caja que
      libera
82     MPI_Status estado;
83
84     while (true) {
85         MPI_Recv(&valor, 1, MPI_INT, MPI_ANY_SOURCE, MPI_ANY_TAG,
86                 MPI_COMM_WORLD, &estado); // recibe el mensaje
87
88         //en este caso se pedia que si habia cajas libres que la etiqueta
      fuese cualquiera, pero he decidido hacerlo de esta manera para
      hacerlo mas simple, aunque en el examen se pedía de esa manera.
89
90         if (estado.MPI_TAG == peticion_pago) {
91             int primera_caja_libre = caja_libre();
92             caja_vacia[primera_caja_libre] = false;
93             cout << "Encargado: asigna caja " << primera_caja_libre << " al
      cliente " << valor << endl;
94             MPI_Ssend(&primera_caja_libre, 1, MPI_INT, estado.MPI_SOURCE,
      0, MPI_COMM_WORLD);
95         }
96         else if (estado.MPI_TAG == peticion_fin_pago) {
97             caja_a_liberar = valor; // el valor recibido es la caja a
      liberar
98             caja_vacia[caja_a_liberar] = true;
99             cout << "Encargado: liberada caja " << caja_a_liberar << " por
      el cliente " << estado.MPI_SOURCE << endl;
100         }
101     }
102 }
103
104 int main( int argc, char** argv )
105 {
106     int id_propio, num_procesos_actual ;
107
108     MPI_Init( &argc, &argv );
109     MPI_Comm_rank( MPI_COMM_WORLD, &id_propio );
110     MPI_Comm_size( MPI_COMM_WORLD, &num_procesos_actual );
111
112     if(num_procesos == num_procesos_actual){
113         if(id_propio < clientes){
114             funcion_cliente(id_propio);
115         }
116         else if(id_propio == id_proceso_encargado){
117             funcion_encargado();
118         }
119     }
120     else
121     {
122         if ( id_propio == 0 ) // solo el primero escribe error, indep. del
      rol
123         { cout << "el número de procesos esperados es: " << num_procesos
      << endl

```



```
124         << "el número de procesos en ejecución es: " <<  
125             num_procesos_actual << endl  
126         << "(programa abortado)" << endl ;  
127     }  
128 }  
129 MPI_Finalize( );  
130 return 0;  
131 }
```

6 Materiales

Los materiales se encuentran en mi página web (pincha aqui.)